CHAMVORY

Sistema de Chamados — Helpdesk

DOCUMENTAÇÃO DE REQUISITOS E TESTES

SENAI · Técnico em Desenvolvimento de Sistemas
Atividade Prática · Turma DESI 2025/1 N1
Data do enunciado: 10/11/2026

Documento consolidado com base no enunciado da atividade e na análise do código-fonte fornecido.

Sumário

1. Apresentação e escopo
2. Perfis e funcionalidades
3. Requisitos funcionais
4. Regras de negócio
5. Requisitos não funcionais
6. Modelo de dados
7. Casos de teste
8. Matriz de rastreabilidade e pendências

1. Apresentação e escopo

O Chamvory centraliza a abertura, consulta e gestão de chamados de suporte técnico. O sistema contempla os perfis Solicitante e Técnico, autenticação, persistência MySQL e API REST. A entrega foi analisada a partir do arquivo compactado do projeto e do enunciado oficial da atividade.

Objetivo

Permitir que solicitantes registrem e acompanhem problemas e que técnicos assumam, atualizem e finalizem os atendimentos, mantendo histórico e controle de acesso.

Tecnologias observadas

Frontend: React 19, TypeScript, Vite 8, Tailwind CSS 4, React Router DOM 7, Axios e Lucide React. Backend: Node.js, TypeScript, Express 5, MySQL/mysql2, JWT, bcrypt, dotenv e Zod.

2. Perfis e funcionalidades

Solicitante: autenticar-se, abrir chamados, consultar os próprios registros, consultar detalhes e histórico e excluir chamados próprios abertos sem histórico. A API também oferece edição de chamados abertos, mas a interface de edição não foi encontrada no frontend entregue.

Técnico: visualizar chamados, consultar fila, assumir chamados disponíveis, registrar observações e finalizar com solução. A tela específica “Meus atendimentos” não foi encontrada no frontend entregue.

3. Requisitos funcionais

RF001 — Autenticar usuário

Perfil: Todos. Permitir autenticação por e-mail e senha.
Situação: Implementado no backend e integrado no frontend.

RF002 — Validar credenciais

Perfil: Todos. Validar campos e rejeitar credenciais inválidas sem liberar a sessão.
Situação: Implementado; resposta de erro permanece na tela de login.

RF003 — Manter sessão

Perfil: Todos. Consultar usuário autenticado ao iniciar e permitir logout com revogação do token.
Situação: Implementado com JWT; revogação em memória no backend.

RF004 — Controlar acesso

Perfil: Todos. Proteger rotas internas e restringir recursos de técnico por perfil.
Situação: Frontend PrivateRoute e middlewares backend.

RF005 — Exibir usuário

Perfil: Todos. Exibir nome, e-mail e perfil do usuário autenticado.
Situação: Implementado na estrutura de perfil/header.

RF006 — Exibir indicadores

Perfil: Todos. Contabilizar chamados por status; solicitante vê seus registros e técnico vê todos.
Situação: Implementado em dashboard.service.

RF007 — Listar chamados

Perfil: Todos. Listar chamados do banco, com escopo conforme o perfil.
Situação: Backend filtra solicitante; técnico recebe todos.

RF008 — Pesquisar chamados

Perfil: Todos. Pesquisar por número ou assunto.
Situação: API aceita query busca; frontend filtra a lista carregada.

RF009 — Consultar detalhes

Perfil: Todos. Exibir dados e histórico de um chamado respeitando permissões.
Situação: Implementado no backend e nas telas de detalhes/atendimento.

RF010 — Abrir chamado

Perfil: Solicitante. Criar chamado com assunto, descrição, categoria e prioridade.
Situação: Implementado e integrado.

RF011 — Gerar identificação

Perfil: Sistema. Gerar número único, data de abertura, solicitante autenticado e status Aberto.
Situação: Implementado no backend.

RF012 — Editar chamado

Perfil: Solicitante. Permitir edição do chamado próprio enquanto Aberto.
Situação: API implementada; interface de edição não identificada no frontend entregue.

RF013 — Excluir chamado

Perfil: Solicitante. Excluir chamado próprio Aberto e sem histórico, após confirmação.
Situação: Backend implementado; tela mostra ação de exclusão, mas confirmar UX deve ser validado.

RF014 — Listar fila

Perfil: Técnico. Listar chamados em ordem cronológica crescente de abertura.
Situação: Backend usa Merge Sort estável; fila frontend consome a API.

RF015 — Assumir chamado

Perfil: Técnico. Assumir chamado Aberto sem técnico e alterar para Em atendimento.
Situação: Implementado com transação e atualização condicional.

RF016 — Registrar observação

Perfil: Técnico responsável. Registrar observação por técnico responsável.
Situação: Implementado; data/hora pode ser enviada pela API.

RF017 — Informar data/hora

Perfil: Técnico responsável. Usar data/hora atual por padrão e permitir ajuste dentro do intervalo válido.
Situação: Backend valida data opcional; campo de edição não foi identificado na interface entregue.

RF018 — Finalizar chamado

Perfil: Técnico responsável. Finalizar atendimento exigindo descrição da solução.
Situação: Implementado no backend e formulário frontend.

RF019 — Preservar histórico

Perfil: Sistema. Criar novo histórico em atribuição, observação e finalização.
Situação: Implementado com inserções transacionais.

RF020 — Bloquear alterações inválidas

Perfil: Sistema. Impedir alterações após finalização, por técnico não responsável ou atribuição duplicada.
Situação: Implementado no backend.

RF021 — Listar categorias

Perfil: Autenticado. Disponibilizar categorias cadastradas para abertura de chamado.
Situação: Implementado via GET /categories.

RF022 — Exibir fila e atendimentos

Perfil: Técnico. Disponibilizar fila e listagem dos atendimentos atribuídos ao técnico.
Situação: Fila existe; tela/rota específica “Meus atendimentos” não identificada.

4. Regras de negócio

RN001 — Escopo do solicitante: O solicitante só consulta e administra chamados próprios.

RN002 — Edição: Somente chamados próprios com status ABERTO podem ser editados.

RN003 — Exclusão: Somente chamados próprios ABERTO e sem histórico podem ser excluídos.

RN004 — Atribuição única: Um chamado só pode ser assumido se estiver ABERTO e sem técnico responsável.

RN005 — Mudança ao assumir: Ao assumir, o técnico autenticado é vinculado e o status passa a EM_ATENDIMENTO.

RN006 — Responsabilidade: Somente o técnico atribuído pode registrar atualizações ou finalizar.

RN007 — Solução obrigatória: A finalização exige descrição da solução.

RN008 — Imutabilidade: Chamados FINALIZADO não aceitam novas alterações.

RN009 — Histórico append-only: Cada atribuição, observação ou finalização gera novo registro sem apagar os anteriores.

RN010 — Data de atendimento: A data deve ser igual ou posterior à abertura e não pode ultrapassar o momento atual.

RN011 — Autenticação: As rotas privadas exigem token válido.

RN012 — Autorização no servidor: O backend valida perfil, propriedade e técnico responsável independentemente da interface.

RN013 — Proteção de senha: Senhas são armazenadas com bcrypt, nunca em texto puro.

RN014 — Ordenação: A listagem da fila é crescente por data de abertura, com id como desempate, usando Merge Sort.

RN015 — Número único: O número do chamado é gerado no servidor no formato CH-000001, derivado do identificador.

5. Requisitos não funcionais

RNF001 — Segurança de credenciais: Senhas devem ser armazenadas com hash bcrypt.
Situação: Implementado no backend.

RNF002 — Autenticação stateless: A API deve autenticar requisições por JWT.
Situação: Implementado; segredo configurado por variável de ambiente.

RNF003 — Autorização: A API deve aplicar controle de acesso por perfil e propriedade.
Situação: Implementado em middlewares e serviços.

RNF004 — Integridade: Relacionamentos devem ser protegidos por PK/FK e transações nas operações críticas.
Situação: Implementado no schema e serviços transacionais.

RNF005 — Usabilidade: A interface deve apresentar formulários claros, feedback e navegação consistente.
Situação: Parcial; validar em execução e completar telas pendentes.

RNF006 — Responsividade: As interfaces devem adaptar-se a desktop e dispositivos menores.
Situação: Há classes responsivas no frontend; não foi realizado teste visual automatizado.

RNF007 — Manutenibilidade: Separar responsabilidades em rotas, controllers, services e repositories.
Situação: Implementado no backend.

RNF008 — Compatibilidade: Executar com Node.js 18+ e MySQL 8+, conforme README.
Situação: Declarado no projeto; ambiente não foi provisionado nesta análise.

RNF009 — Disponibilidade de sessão: Logout deve invalidar o token atual.
Situação: Implementado com lista de revogação em memória; reinício do processo limpa a lista.

RNF010 — Desempenho da fila: Ordenar registros com algoritmo de complexidade O(n log n).
Situação: Merge Sort implementado; benchmark não realizado.

6. Modelo de dados

O banco helpdesk_db possui quatro tabelas: usuarios, categorias, chamados e historico_atendimentos. As tabelas incluem chaves primárias e estrangeiras; o número do chamado e o e-mail do usuário são únicos. O relacionamento de técnico responsável é opcional em chamados (FK nullable). Consulte o arquivo DER.png para atributos e cardinalidades.

Cardinalidades principais

Usuário solicitante 1:N Chamados; Categoria 1:N Chamados; Usuário técnico 1:N Chamados (opcional no lado do chamado); Chamado 1:N Histórico; Usuário técnico 1:N Histórico.

População do banco

O script helpdesk_db.sql cria as quatro tabelas e popula 4 usuários, 3 categorias, 4 chamados e 4 registros de histórico, atendendo ao mínimo de três registros por tabela. As senhas de demonstração são armazenadas como hashes bcrypt. O script apaga e recria helpdesk_db; execute somente após backup.

7. Casos de teste

Ambiente previsto: Node.js 18+ (recomendado 20+), MySQL 8+, navegador moderno, API local e cliente HTTP (Postman/Insomnia). Os resultados esperados abaixo são critérios de aceitação. Nenhum teste funcional foi executado durante a geração deste pacote; portanto, o resultado obtido permanece “Não executado”.

CT001 — Login com credenciais válidas

Requisito: RF001/RF002
Pré-condições: Usuário de demonstração existente.
Passos: Enviar e-mail e senha corretos no formulário de login.
Esperado: Sessão criada e redirecionamento conforme perfil.
Obtido: Não executado
Observação: Requer API e MySQL ativos.

CT002 — Login com credenciais inválidas

Requisito: RF001/RF002
Pré-condições: Tela de login acessível.
Passos: Enviar senha incorreta para e-mail cadastrado.
Esperado: API rejeita; usuário permanece sem sessão e recebe mensagem de erro.
Obtido: Não executado
Observação: Requer API e MySQL ativos.

CT003 — Acesso sem autenticação

Requisito: RF004
Pré-condições: Token ausente e sessão limpa.
Passos: Abrir diretamente /dashboard e /tecnico/fila.
Esperado: Redirecionamento para /login.
Obtido: Não executado
Observação: Requer teste no navegador.

CT004 — Logout e bloqueio posterior

Requisito: RF003/RF004
Pré-condições: Usuário autenticado.
Passos: Executar Sair e tentar abrir rota privada.
Esperado: Token revogado/removido e acesso bloqueado.
Obtido: Não executado
Observação: Requer API e navegador.

CT005 — Abrir chamado válido

Requisito: RF010/RF011
Pré-condições: Solicitante autenticado e categoria cadastrada.
Passos: Preencher assunto, descrição, categoria e prioridade; enviar.
Esperado: Registro criado com CH-..., solicitante atual, data atual e ABERTO.
Obtido: Não executado
Observação: Requer API e MySQL.

CT006 — Abertura com campos ausentes

Requisito: RF010
Pré-condições: Solicitante autenticado.
Passos: Enviar formulário sem assunto ou descrição/categoria/prioridade.
Esperado: Validação impede envio e informa campos inválidos.
Obtido: Não executado
Observação: Requer navegador.

CT007 — Buscar por número

Requisito: RF008
Pré-condições: Existirem chamados cadastrados.
Passos: Consultar por CH-000001.
Esperado: Retornar somente registros correspondentes.
Obtido: Não executado
Observação: Requer API e dados seed.

CT008 — Buscar por assunto

Requisito: RF008
Pré-condições: Existirem chamados cadastrados.
Passos: Consultar parte do assunto de um chamado.
Esperado: Retornar registros correspondentes ao texto.
Obtido: Não executado
Observação: Requer API e dados seed.

CT009 — Editar chamado próprio aberto

Requisito: RF012
Pré-condições: Solicitante dono; chamado ABERTO.
Passos: Enviar PUT /tickets/:id com um campo válido.
Esperado: Dados atualizados.
Obtido: Não executado
Observação: Endpoint existe; UI de edição ausente.

CT010 — Editar chamado em atendimento

Requisito: RF012/RN002
Pré-condições: Solicitante dono; status EM_ATENDIMENTO.
Passos: Enviar PUT /tickets/:id.
Esperado: API rejeita com conflito (409).
Obtido: Não executado
Observação: Requer API e dados preparados.

CT011 — Excluir chamado sem histórico

Requisito: RF013
Pré-condições: Solicitante dono; ABERTO; sem histórico.
Passos: Confirmar exclusão pela interface/API.
Esperado: Registro excluído.
Obtido: Não executado
Observação: Requer API e dados preparados.

CT012 — Excluir chamado com histórico

Requisito: RF013/RN003
Pré-condições: Solicitante dono; ABERTO; possui histórico.
Passos: Enviar DELETE /tickets/:id.
Esperado: API rejeita exclusão (409).
Obtido: Não executado
Observação: Requer API e dados preparados.

CT013 — Solicitante consultar chamado alheio

Requisito: RF007/RN001
Pré-condições: Dois solicitantes cadastrados.
Passos: Autenticar como A e consultar id de B.
Esperado: Chamado não é revelado; serviço responde 404 na consulta.
Obtido: Não executado
Observação: Requer API e usuários seed.

CT014 — Solicitante acessar gestão técnica

Requisito: RF004/RN012
Pré-condições: Autenticar como solicitante.
Passos: Acessar /tecnico/fila e endpoint protegido.
Esperado: Frontend redireciona e backend rejeita perfil.
Obtido: Não executado
Observação: Requer navegador e API.

CT015 — Técnico assumir chamado disponível

Requisito: RF015
Pré-condições: Técnico autenticado; chamado ABERTO sem técnico.
Passos: Enviar PATCH /tickets/:id/assign.
Esperado: Técnico vinculado, status EM_ATENDIMENTO e histórico criado.
Obtido: Não executado
Observação: Requer API e MySQL.

CT016 — Assumir chamado já atribuído

Requisito: RF015/RN004
Pré-condições: Chamado com técnico_id preenchido.
Passos: Outro técnico tenta assumir.
Esperado: API rejeita com conflito; responsável original permanece.
Obtido: Não executado
Observação: Requer API e dois técnicos.

CT017 — Registrar observação

Requisito: RF016/RN006
Pré-condições: Técnico responsável; chamado em atendimento.
Passos: Enviar observação pelo formulário.
Esperado: Novo histórico criado e histórico anterior preservado.
Obtido: Não executado
Observação: Requer API e MySQL.

CT018 — Consultar histórico

Requisito: RF009/RF019
Pré-condições: Chamado com registros de histórico.
Passos: Abrir detalhes/histórico.
Esperado: Registros exibidos em ordem cronológica e conforme permissão.
Obtido: Não executado
Observação: Requer API e navegador.

CT019 — Data anterior à abertura

Requisito: RF017/RN010
Pré-condições: Chamado atribuído; data de abertura conhecida.
Passos: Enviar data_atendimento anterior à abertura.
Esperado: API rejeita com erro de data inválida.
Obtido: Não executado
Observação: Requer API.

CT020 — Data futura

Requisito: RF017/RN010
Pré-condições: Chamado atribuído.
Passos: Enviar data_atendimento posterior ao momento atual.
Esperado: API rejeita com erro de data inválida.
Obtido: Não executado
Observação: Requer API.

CT021 — Finalizar com solução

Requisito: RF018/RN007
Pré-condições: Técnico responsável; chamado em atendimento.
Passos: Enviar PATCH /finish com solução preenchida.
Esperado: Status FINALIZADO e histórico de solução criado.
Obtido: Não executado
Observação: Requer API e MySQL.

CT022 — Finalizar sem solução

Requisito: RF018/RN007
Pré-condições: Técnico responsável; chamado em atendimento.
Passos: Enviar PATCH /finish sem solução.
Esperado: Validação rejeita a operação.
Obtido: Não executado
Observação: Requer API.

CT023 — Alterar chamado finalizado

Requisito: RF020/RN008
Pré-condições: Chamado FINALIZADO.
Passos: Tentar adicionar histórico ou finalizar novamente.
Esperado: API rejeita alteração.
Obtido: Não executado
Observação: Requer API e dados preparados.

CT024 — Atualizar como técnico não responsável

Requisito: RF020/RN006
Pré-condições: Chamado atribuído ao técnico A; autenticar técnico B.
Passos: Enviar observação ou finalizar.
Esperado: API rejeita por técnico não responsável.
Obtido: Não executado
Observação: Requer API e dois técnicos.

CT025 — Ordenação cronológica da fila

Requisito: RF014/RN014
Pré-condições: Lista com datas fora de ordem e empates.
Passos: Executar mergeSort e comparar resultado crescente.
Esperado: Mais antigo primeiro; id desempata; algoritmo mantém estabilidade.
Obtido: Não executado nesta sessão
Observação: Existe teste automatizado em backend/tests/merge-sort.test.ts; não foi executado aqui.

8. Matriz de rastreabilidade e pendências

A matriz completa está em docs/matriz_rastreabilidade.md. Principais pendências identificadas: (1) falta de tela/ação de edição no frontend, embora exista PUT no backend; (2) falta de rota/tela “Meus atendimentos”; (3) falta de campo de data/hora editável na interface de atendimento, embora a API aceite e valide data_atendimento; (4) revogação JWT mantida apenas em memória; (5) edição de perfil não disponível na API.

Verificação da execução

O pacote original contém código-fonte e artefatos compilados (dist), além de um teste automatizado para Merge Sort. Os comandos de build, lint, npm test e os testes de integração não foram executados nesta sessão. Não há declaração de aprovação de testes sem evidência.

Arquivos complementares

docs/requisitos_funcionais.md · docs/regras_de_negocio.md · docs/requisitos_nao_funcionais.md · docs/casos_de_teste.md · docs/documentacao_tecnica.md · docs/matriz_rastreabilidade.md · docs/fluxos/